Profiling dan Optimasi ORM

Query Profiling

Kita akan menggunakan library django silk untuk mengecek performa & query, langkah instalasi & setupnya adalah sebagai berikut:

- Tambahkan requirement : `django-silk==5.1.0`
- Pada file `settings.py` kita ubah di dua konfigurasi yaitu `MIDDLEWARE` dan `INSTALLED_APPS`:

```
MIDDLEWARE = [
    ...
    'silk.middleware.SilkyMiddleware',
]

INSTALLED_APPS = (
    ...
    'silk'
)
```

- Pada file `urls.py` kita menambahkan url Silk untuk dapat melihat halaman profiler-nya

```
urlpatterns += [path('silk/', include('silk.urls', namespace='silk'))]
```

- Rebuild image dengan `docker-compose down` dan `docker-compose up -d --build`
- Migrasikan tabel silk dengan perintah

```
docker exec simple_lms python manage.py makemigrations
```

- Django silk akan otomatis melakukan profiling terhadap halaman yang diakses user, dan untuk melihat hasil profiling-nya kita bisa membuat `localhost:8001/silk`

N+1 Problems

- Persiapkan data dummy untuk menunjukkan N+1 Problems

Query yang Memerlukan Relasi ke Beberapa Tabel:

- **Statistik Semua Course**:
  - jumlah semua course,
  - course dengan harga tertinggi,
  - course dengan harga terendah,
  - course dengan jumlah member terbanyak,
  - course dengan jumlah member paling sedikit

```
def courseStat(request):
  courses = Course.objects.all()
```

```
  stats = courses.aggregate(max_price=Max('price'),
                            min_price=Min('price'),
                            avg_price=Avg('price'))
  cheapest = Course.objects.filter(price=stats['min_price'])
  expensive = Course.objects.filter(price=stats['max_price'])
  popular = Course.objects.annotate(member_count=Count('coursemember'))\
                          .order_by('-member_count')[:5]
  unpopular = Course.objects.annotate(member_count=Count('coursemember'))\
                          .order_by('member_count')[:5]

  result = {'course_count': len(courses), 'courses': stats,
            'cheapest': serializers.serialize('python', cheapest),
            'expensive': serializers.serialize('python', expensive),
            'popular': serializers.serialize('python', popular),
            'unpopular': serializers.serialize('python', unpopular)}
  return JsonResponse(result, safe=False)
```

- **Detail Statistik Course**:
  - Detail course
  - Jumlah member
  - jumlah konten
  - jumlah semua komentar
  - konten yang paling banyak dikomentari

```
def courseDetail(request, course_id):
   course = Course.objects.annotate(member_count=Count('coursemember'),
                                    content_count=Count('coursecontent'),
                                    comment_count=Count('coursecontent__comment'))\
                          .get(pk=course_id)
   contents = CourseContent.objects.filter(course_id=course.id)\
               .annotate(count_comment=Count('comment'))\
               .order_by('-count_comment')[:3]
   result = {"name": course.name, 'description': course.description, 'price': course.price
             'member_count': course.member_count, 'content_count': course.content_count,
             'teacher': {'username': course.teacher.username, 'email':
                         course.teacher.email, 'fullname': course.teacher.first_name},
             'comment_stat': {'comment_count': course.comment_count,
                              'most_comment':[{'name': content.name,
                                               'comment_count': content.count_comment}
                                               for content in contents]},
             }

   return JsonResponse(result)
```

- [ ] **Statistik semua user**:
  - Jumlah semua user yang bukan admin
  - Jumlah user yang membuat course
  - Jumlah user yang tidak memiliki course
  - Rata2 jumlah course yang diikuti 1 user
  - User yang mengikuti course terbanyak
  - List user yang tidak mengikuti course sama sekali

- [ ] **Detail Statistik User**:
  - Detail user
  - Jumlah course yang diikuti
  - Jumlah course yang dibuat
  - Jumlah member dari semua course yang dibuat
  - Jumlah komentar yang diposting

Database access optimization

- Select related
- Select prefetch
- Input data massal menggunakan `bulk_create`

```
with open(filepath+'contents.json') as jsonfile:
    comments = json.load(jsonfile)
    obj_create = []
    for num, row in enumerate(comments):
        if not CourseContent.objects.filter(pk=num+1).exists():
            obj_create.append(CourseContent(
                        course_id=Course.objects.get(pk=int(row['course_id'])),
                         video_url=row['video_url'], name=row['name'],
                         description=row['description'], id=num+1))
    CourseContent.objects.bulk_create(obj_create)


with open(filepath+'comments.json') as jsonfile:
    comments = json.load(jsonfile)
    obj_create = []
    for num, row in enumerate(comments):
        if int(row['user_id']) > 50:
            row['user_id'] = randint(5, 40)
        if not Comment.objects.filter(pk=num+1).exists():
            obj_create.append(Comment(
                         content_id=CourseContent.objects.get(pk=int(row['content_id'])),
                        user_id=User.objects.get(pk=int(row['user_id'])), id=num+1,
                        comment=row['comment']))
    Comment.objects.bulk_create(obj_create)
```